

START HERE — Project APEX Engineering Apprenticeship

The route from beginner builder to independent world-model researcher

OpenAI

August 2026

Table of Contents

1 What this package is

This package is designed to correct a common failure in technical textbooks: naming concepts without constructing understanding. The primary teaching core does not ask you to memorize GRUs, state-space models, RSSMs, Dreamer, JEPA or Airflow. It creates the problem that each idea solves, makes you build the simplest previous solution, exposes its failure, and only then introduces the stronger mechanism.

The endpoint is a working Project APEX V1: an F1 telemetry world-simulation engine that can use synthetic evidence offline and canonical data produced by FastF1 or OpenF1 adapters, train deterministic and latent sequence models, generate counterfactual rollouts, evaluate them by horizon, register artifacts and serve an interactive race-engineering interface.

The deeper endpoint is independence. You should be able to read a world-model paper, reconstruct its mechanism, identify its assumptions, implement a small reproduction, design a matched APEX ablation, and reject the idea when evidence does not support it.

2 The correct reading and building order

2.1 Stage 1 — Teaching core

Open textbook/output/Project_APEX_Engineering_Apprenticeship_Teaching_Core.pdf.

Work through Chapters 1–20 in order. Every chapter has a runnable lab. Do not read the solutions section before writing your prediction. When a chapter names an APEX source path, open that file only after understanding the smaller lab.

The core sequence is:

Simulator before neural network.

State, action, context and partial observability.

Units, forces and integration.

Sampling, aliasing and alignment.

Canonical contracts and quality gates.

FastF1/OpenF1 source adapters.
 Windows, splits, normalization and leakage.
 Statistics, baselines and uncertainty.
 Tensors, autograd and training.
 One-step transitions and autoregressive rollout.
 GRU memory.
 Classical state-space models.
 Selective SSMs and Mamba concepts.
 Autoencoders and variational latent state.
 RSSMs and Dreamer.
 JEPA, LeWorldModel and fine-tuning.
 Evaluation, ablation and OOD tests.
 MPC and CEM planning.
 Pipelines, Airflow, registry and monitoring.
 Complete APEX V1 and research roadmap.

2.2 Stage 2 — Executable lab manual

Use Project_APEX_Executable_Lab_Manual.pdf beside the labs/ folder. The manual contains exact execution commands, expected output captured from the packaged solution, debugging instructions and extension work.

A lab is complete only when:

- you predicted output before running;
- you can explain every axis and unit;
- you recomputed one result by hand;
- you injected the listed defect;
- your regression test fails on the defect;
- your repair passes the test;
- you implemented the extension independently.

2.3 Stage 3 — Ten-project ladder

Use Project_APEX_Ten_Project_Instructor_Ladder.pdf and build Projects 1–9 before treating the production engine as your own work.

The projects are not copy exercises. Hide or rename the supplied implementation, write acceptance tests, make your own attempt, then compare. Each project ends with an Architecture Decision Record.

2.4 Stage 4 — Production debugging casebook

Use Project_APEX_Production_Debugging_Casebook.pdf with debugging_cases/.

For each incident, stop before the root-cause section. Preserve a failing artifact, identify the earliest broken contract, implement a probe, repair the cause, and add a test. The cases cover units, axes, leakage, temporal joins, normalizers, recurrent state, eval mode, gradients, SSM stability, KL collapse, planner exploitation, artifact lineage, Airflow payloads and UI false precision.

2.5 Stage 5 — Research reading and design

Use `research_reading/Project_APEX_Research_Reading_and_Design_Guide.pdf` while reading primary papers. Draw training and imagination graphs separately. Mark gradient and stop-gradient paths. Reproduce the smallest mechanism before adapting it.

2.6 Stage 6 — Field workbook

The workbook is not optional. Use it for predictions, manual calculations, ablation manifests, incident reports, architecture decisions, paper maps and independent-build checkpoints.

2.7 Stage 7 — Annotated source companion

The source companion contains line-numbered production code explanations. Use it after attempting to trace the code yourself. It is a verification tool, not a substitute for reading source.

2.8 Stage 8 — Production capstone

Follow `apex_engine/START_HERE.md`, run tests, execute a fresh run, inspect artifacts, launch the UI and reproduce one scenario outside the UI. Then replace one module with your own implementation under the same tests and evaluation.

3 Recommended 24-week apprenticeship

3.1 Weeks 1-4: causal foundations

Complete Chapters 1-5, Labs 1-4 and Projects 1-3. Spend more time on units, time, contracts and causal slicing than on model code. These layers determine whether later metrics mean anything.

Deliverables:

- one-dimensional integrator;
- causal lap sandbox;
- canonical schema;
- validation suite;
- numerical-method decision record.

3.2 Weeks 5-8: evidence and statistical controls

Complete Chapters 6-10, Labs 5-10 and Projects 4-5. Ingest or mock both public data sources, create session splits, build baselines and trace a full training step.

Deliverables:

- adapter fixtures;
- leakage audit;
- persistence/linear baselines;
- residual report;
- instrumented training trace.

3.3 Weeks 9-12: deterministic sequence dynamics

Complete Chapters 11-13, Labs 11-14 and Projects 6-7. Build GRU and SSM models under matched budgets. Do not move on until you can explain hidden-state lifetime and stability.

Deliverables:

GRU horizon curves;
selective-memory diagnostic;
hidden-norm plot;
matched architecture decision.

3.4 Weeks 13–16: latent world models

Complete Chapters 14–16, Labs 15–17 and Project 8. Build deterministic compression, VAE sampling and RSSM prior/posterior paths. Read DreamerV3 and I-JEPA using the research guide.

Deliverables:

reconstruction and future probes;
KL diagnostics;
posterior versus prior rollout comparison;
representation-collapse test.

3.5 Weeks 17–19: planning and evaluation

Complete Chapters 17–18, Lab 18 and Project 9. Build a CEM planner and deliberately give it a model/reward loophole. Define promotion and abstention rules before comparing models.

Deliverables:

full evaluation matrix;
planner-exploitation incident report;
uncertainty/OOD constraint;
planning decision record.

3.6 Weeks 20–22: production engineering

Complete Chapter 19, Labs 19–20 and all debugging cases. Run the local pipeline twice, simulate stage failure, inspect the registry and compile the Airflow DAG.

Deliverables:

immutable run artifacts;
resumable failure test;
lineage report;
monitoring specification.

3.7 Weeks 23–24: APEX capstone and independent research

Complete Chapter 20 and Project 10. Launch the UI, reproduce one plotted point from source frames, run a canonical-source experiment and write an original architecture proposal.

Deliverables:

fresh verified APEX run;
UI-to-model provenance trace;
model card with trusted horizon;
original research hypothesis and ablation plan.

4 Environment setup

```
cd Project_APEX_Engineering_Apprenticeship
python -m venv .venv
source .venv/bin/activate
```

```
cd apex_engine
pip install -e .
pytest -q
```

Run one fast reference:

```
apexsim run \
  --config configs/fast.yaml \
  --run-id my_apprenticeship_reference
```

Launch the UI:

```
apexsim ui \
  --run-dir artifacts/runs/my_apprenticeship_reference \
  --config configs/fast.yaml
```

For a lab:

```
cd ../labs/10_training_loop
python solution.py
```

5 Evidence standards

Do not claim a model is better because its architecture is newer. Every comparison should preserve:
canonical dataset and version;

split identifiers;

normalization;

history and horizon;

training budget;

seed set;

evaluation implementation;

parameter/latency context;

physical guardrails.

Do not claim a system is production ready because it has an API or Dockerfile. Production evidence includes reproducible artifacts, retries, lineage, monitoring, rollback, security boundaries and product-level limitations.

Do not claim a model is Dreamer because it contains an RSSM. Dreamer includes world-model learning, reward/continuation prediction and behaviour learning from imagination.

Do not claim a toy selective recurrent cell is Mamba. Use it to understand input-dependent memory, then evaluate the complete official architecture when scale and sequence length justify it.

6 Mastery checkpoints

You are ready to progress when you can answer and demonstrate:

What exactly is the state, action, context and target?
Which input values are available at prediction time?
What is the split unit and why?
Where are normalization statistics fitted and stored?
What does each tensor axis represent?
What changes after backward and after optimizer step?
How is predicted state fed into the next rollout step?
When is hidden state reset?
Why is an SSM stable or unstable?
What differs between RSSM prior and posterior?
How do you detect collapse?
What baseline must a new model beat?
What horizon is trusted and why?
What can a scenario user change?
How do artifacts trace back to data/config/code?
What failure motivates your proposed architecture?

7 Final independent test

Start an empty repository. Rebuild a smaller APEX using only:
canonical schema;
acceptance tests;
system architecture diagram;
evaluation requirements.

Do not copy implementation. When complete, compare with the supplied repository and write a postmortem: decisions you made differently, defects you avoided or introduced, and one research question created by the experience.

The curriculum succeeds when the supplied code stops being the authority and becomes one implementation you can critique.